

Full Stack Development

Solved Question Paper — Mid Term (Odd) Semester Examination, October 2024
MCA 1st Semester | Paper Code: TMC 101 | Graphic Era Hill University | Max. Marks: 50

Note: Each question is answered in full (both the (a) and (b) options), even though the exam only requires one, so this document doubles as a complete revision set for HTML/CSS fundamentals covered in the syllabus.

Q1 (a)

10 Marks | CO1

List out the advantages and disadvantages of XHTML over HTML? Compare and contrast with proper example.

Answer:

XHTML (Extensible HyperText Markup Language) is a stricter, XML-based reformulation of HTML. While HTML is forgiving about syntax (browsers can guess what a malformed tag meant), XHTML must obey strict XML rules — every tag must be properly closed, correctly nested, and written in lowercase. This makes XHTML far more rigid than classic HTML, but also more predictable and machine-readable.

Advantages of XHTML over HTML

- **Strict, well-formed syntax:** every element must be properly closed and nested, which enforces clean, consistent code.
- **XML compatibility:** because it is XML-based, XHTML documents can be parsed by any standard XML parser and easily combined with other XML technologies (XSLT, RSS, MathML, SVG).
- **Better cross-platform support:** works consistently across browsers, mobile devices and non-browser XML tools, since it follows one strict specification.
- **Improved error handling:** because the syntax is strict, errors are caught early rather than being silently 'fixed' by the browser, which improves code quality and maintainability.
- **Extensibility:** custom XML namespaces/modules can be mixed into a document, useful for embedding specialised markup.
- **Better for accessibility & automated tools:** the predictable, well-formed structure makes it easier for screen readers and parsers to process the document.

Disadvantages of XHTML compared to HTML

- **Strictness is unforgiving:** a single unclosed tag or syntax mistake can break the entire page, whereas HTML browsers simply try to recover.
- **Steeper learning curve:** developers must be more careful and disciplined about writing valid markup.
- **Less browser leniency:** if served with the correct XML MIME type, some older browsers render a blank page (or a parse error) on any invalid markup.
- **Slower adoption / largely superseded:** HTML5 absorbed most of XHTML's clean-syntax benefits while keeping HTML's error tolerance, so XHTML is rarely used for new projects today.
- **Extra verbosity:** attributes must always have quoted values and elements like `<br>` must be self-closed (`<br />`), which adds a little markup overhead.

Comparison Table: HTML vs XHTML

Aspect	HTML	XHTML
Syntax strictness	Lenient — browsers auto-correct many mistakes	Strict — must be well-formed XML; a single error can break the page
Tag closing	Some tags may be left unclosed, e.g. <code>
</code> , <code><p></code>	Every tag must be explicitly closed, e.g. <code>
</code> , <code><p>...</p></code>
Case sensitivity	Tag/attribute names are case-insensitive	Tag/attribute names must be lowercase

Aspect	HTML	XHTML
Attribute values	Quotes around attribute values are often optional	Attribute values must always be quoted, e.g. width="100"
Nesting	Improper nesting is often tolerated	Elements must be properly nested, e.g. <i>text</i>
Root document type	Plain text/markup, not XML	Must be valid XML, can use an XML declaration
Error handling	Browser silently repairs errors and keeps rendering	Parsing stops / page may fail to render on error

Example: Same Content in HTML vs XHTML

HTML (tolerant of unclosed tags, uppercase, unquoted attributes):

```
<HTML>
<BODY>
<p>This is a paragraph
<br>
<img src=photo.jpg width=200>
</BODY>
</HTML>
```

Equivalent, strict XHTML (all lowercase, all tags closed, quoted attributes):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head><title>XHTML Example</title></head>
<body>
  <p>This is a paragraph</p>
  <br />
  
</body>
</html>
```

Design a web page about you. Use CSS to configure a background color and text color for the page. Include the following on your web page: (i) Your name (ii) A description of your favourite hobbies and activities (iii) Academic details in the tabular form (iv) A photo of yourself

Answer:

The program below builds a simple personal webpage that sets a page-wide background color and text color using CSS, and includes a name heading, a hobbies paragraph, an academic-details table, and a placeholder photo — all four required elements.

```
<!DOCTYPE html>
<html>
<head>
  <title>About Me</title>
  <style>
    body {
      background-color: #eaf4fb; /* page background color */
      color: #1a1a2e;           /* page text color */
      font-family: Arial, Helvetica, sans-serif;
      margin: 40px;
    }
    h1 { color: #0b6e4f; }
    table {
      border-collapse: collapse;
      width: 60%;
      margin-top: 10px;
    }
    th, td {
      border: 1px solid #444;
      padding: 8px 12px;
      text-align: left;
    }
    th { background-color: #0b6e4f; color: #fff; }
    img {
      width: 150px;
      border-radius: 8px;
      margin-top: 10px;
      border: 2px solid #0b6e4f;
    }
  </style>
</head>
<body>

  <!-- (i) Name -->
  <h1>Rahul Sharma</h1>

  <!-- (ii) Hobbies and activities -->
  <h2>My Hobbies & Activities</h2>
  <p>
    I am an MCA student who enjoys competitive programming, playing badminton on
    weekends, reading science-fiction novels, and building small web projects to
    practise front-end and back-end development.
  </p>

  <!-- (iii) Academic details in tabular form -->
  <h2>Academic Details</h2>
  <table>
    <tr><th>Qualification</th><th>Institution</th><th>Year</th><th>Percentage/CGPA</th></tr>
    <tr><td>10th</td><td>ABC Public School</td><td>2018</td><td>92%</td></tr>
    <tr><td>12th</td><td>ABC Public School</td><td>2020</td><td>89%</td></tr>
    <tr><td>B.Sc. (CS)</td><td>XYZ College</td><td>2023</td><td>8.4 CGPA</td></tr>
    <tr><td>MCA (Pursuing)</td><td>Graphic Era Hill University</td><td>2024-26</td><td>-</td></tr>
  </table>
```

```
<!-- (iv) Photo -->
<h2>My Photo</h2>


</body>
</html>
```

The <style> block sets the page's background-color and text color on the <body> selector so they apply site-wide; the table uses border-collapse and padding for a clean, readable layout; and the tag displays the photo with a rounded border. Replace 'myphoto.jpg' with the actual filename of your photo stored in the same folder as this HTML file.

Discuss the meaning of hyperlink in HTML? How to create universal linking and target on the web page? Explain with proper example.

Answer:

A **hyperlink** is a reference/link from one web page (or a specific point within it) to another resource — another page, a section of the same page, an email address, a file, or an external website — that the user can activate by clicking. Hyperlinks are what make the web a connected, navigable 'web' of documents rather than a set of isolated pages. In HTML, a hyperlink is created using the anchor tag `<a>` with an `href` attribute pointing to the destination.

Basic Syntax

```
<a href="destination_url" target="_blank">Link Text</a>
```

Types of Hyperlinks

- **External link:** points to another website, e.g. `<a href="https://www.google.com">Google</a>`.
- **Internal/relative link:** points to another page within the same site, e.g. `<a href="about.html">About Us</a>`.
- **Bookmark/anchor link:** jumps to a specific section within the same page using an id and a '#' reference.
- **Email link:** opens the user's mail client, e.g. `<a href="mailto:someone@example.com">Email Us</a>`.
- **Universal (absolute) link:** a complete, fully-qualified URL that works from anywhere, regardless of the current page's location — explained below.

Universal Linking (Absolute URLs)

A **universal link** (absolute URL) specifies the complete address of a resource — protocol, domain, and full path — so it resolves correctly no matter where it is placed, unlike a relative link which depends on the current page's location. It follows the general format:

scheme://host[:port]/path?query#fragment

e.g. `https://www.example.com/courses/mca/notes.html`

This is contrasted with a **relative URL** such as `../notes.html`, which is resolved relative to the current document's folder and therefore breaks if the file structure changes.

The target Attribute

The target attribute controls **where** the linked document opens:

Value	Behaviour
<code>_self</code>	Opens the link in the same frame/tab (default behaviour).
<code>_blank</code>	Opens the link in a new browser tab or window.
<code>_parent</code>	Opens the link in the parent frame of the current frame.
<code>_top</code>	Opens the link in the full body of the window, breaking out of all frames.
<code>framename</code>	Opens the link inside a specific named frame/iframe.

Example Program

```
<!DOCTYPE html>
<html>
<head><title>Hyperlink Demo</title></head>
<body>
```

```
<h2>Hyperlink Examples</h2>

<!-- Universal (absolute) link opening in a NEW tab -->
<p>
  <a href="https://www.wikipedia.org" target="_blank">
    Visit Wikipedia (opens in new tab)
  </a>
</p>

<!-- Relative/internal link opening in the SAME tab -->
<p>
  <a href="contact.html" target="_self">Go to Contact Page</a>
</p>

<!-- Bookmark link to a section further down the same page -->
<p><a href="#section2">Jump to Section 2</a></p>

<h3 id="section2">Section 2</h3>
<p>This is the target section reached via the bookmark link above.</p>

</body>
</html>
```

Create a web page that uses a description list to display three network protocols and their descriptions. Include a hyperlink to a website that provides information about the protocol.

Answer:

HTML provides a **description list** using the <dl> (description list) element, which wraps pairs of <dt> (description term) and <dd> (description detail) elements — ideal for glossaries, FAQs, or, as required here, listing network protocols with their explanations.

```
<!DOCTYPE html>
<html>
<head><title>Network Protocols</title></head>
<body>

  <h2>Common Network Protocols</h2>

  <dl>
    <dt><b>HTTP (HyperText Transfer Protocol)</b></dt>
    <dd>
      Used for transferring web pages and resources between a client (browser)
      and a web server. It is the foundation of data communication on the World
      Wide Web.
      <br>
      <a href="https://developer.mozilla.org/en-US/docs/Web/HTTP" target="_blank">
        Learn more about HTTP
      </a>
    </dd>

    <dt><b>FTP (File Transfer Protocol)</b></dt>
    <dd>
      Used for transferring files between a client and a server over a network,
      commonly used to upload/download files to/from a web server.
      <br>
      <a href="https://en.wikipedia.org/wiki/File_Transfer_Protocol" target="_blank">
        Learn more about FTP
      </a>
    </dd>

    <dt><b>SMTP (Simple Mail Transfer Protocol)</b></dt>
    <dd>
      Used for sending email messages between mail servers and from a mail
      client to a mail server.
      <br>
      <a href="https://en.wikipedia.org/wiki/Simple_Mail_Transfer_Protocol" target="_blank">
        Learn more about SMTP
      </a>
    </dd>
  </dl>

</body>
</html>
```

Each protocol name is wrapped in <dt> (the term) and its explanation in <dd> (the detail); an <a> tag with target="_blank" is embedded inside each <dd> so visitors can open more information about that protocol in a new tab without losing their place on the current page.

List out the various ways to apply CSS styles to a web page. Create a web page which includes following properties on an element: (i) Font color, font size and font weight using inline CSS (ii) Background-image repetition, attachment and position properties using internal CSS (iii) Border, padding and margin properties using External CSS.

Answer:

CSS (Cascading Style Sheets) can be applied to an HTML document in three ways, which differ in where the style rules are written and how broadly they apply:

Method	Description & Where Used
Inline CSS	Styles written directly on a single element using its style attribute. Highest priority (specificity), but not reusable — applies to only that one element.
Internal / Embedded CSS	Styles written inside a <style> block in the <head> of the HTML document. Applies to the whole page but not to other pages.
External CSS	Styles written in a separate .css file and linked using <link rel="stylesheet" href="...">. Best for large or multi-page sites — one file can style every page and is cached by the browser.

Order of Precedence (Cascade)

When multiple methods target the same element, the browser applies them in this order of increasing priority: External CSS < Internal CSS < Inline CSS < !important rule.

Program Demonstrating All Three Methods

index.html — uses inline, internal and external CSS together:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Methods Demo</title>

  <!-- 2. INTERNAL CSS: background-image repetition, attachment, position -->
  <style>
    .bg-box {
      height: 250px;
      background-image: url("pattern.png");
      background-repeat: repeat-x;      /* repeats only horizontally */
      background-attachment: fixed;     /* stays fixed while scrolling */
      background-position: center top;  /* positioned at top-center */
    }
  </style>

  <!-- 3. EXTERNAL CSS: border, padding, margin -->
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <!-- 1. INLINE CSS: font color, size, weight -->
  <p style="color: #b00020; font-size: 20px; font-weight: bold;">
    This text uses inline CSS for color, size and weight.
  </p>

  <div class="bg-box">
    This box uses internal CSS for its background-image properties.
  </div>

  <div class="bordered">
    This box uses external CSS for border, padding and margin.
```

```
</div>
```

```
</body>
```

```
</html>
```

styles.css (external file, linked above):

```
.bordered {  
  border: 3px solid #2c3e50;  
  padding: 20px;  
  margin: 25px;  
  background-color: #eef2f5;  
}
```

Write the HTML and CSS code for an embedded style sheet that configures links without underlines; a background color of white; text color of black; is in Arial, Helvetica, or a sans-serif font; and has a class called new that is bold and italic.

Answer:

An 'embedded' style sheet means the CSS rules are written inside a <style> block in the document's <head> (internal CSS). The code below satisfies every requirement: links with no underline, a white page background, black text, an Arial/Helvetica/sans-serif font stack, and a .new class that renders text bold and italic.

```
<!DOCTYPE html>
<html>
<head>
<title>Embedded Style Sheet Example</title>

<style>
  body {
    background-color: white;
    color: black;
    font-family: Arial, Helvetica, sans-serif;
  }

  a {
    text-decoration: none; /* removes the underline from links */
    color: black;
  }

  .new {
    font-weight: bold;
    font-style: italic;
  }
</style>
</head>

<body>

  <h2>Embedded Style Sheet Demo</h2>

  <p>This paragraph is in Arial/Helvetica/sans-serif, with a white
    background and black text, as configured by the embedded style sheet.</p>

  <p><a href="https://www.example.com">This link has no underline</a></p>

  <p class="new">This paragraph uses the "new" class – bold and italic text.</p>

</body>
</html>
```

The font-family: Arial, Helvetica, sans-serif; declaration is a font stack: the browser tries Arial first, falls back to Helvetica if unavailable, and finally to any generic sans-serif font, guaranteeing a consistent look across systems.

Compare and contrast the pseudo classes and pseudo elements in CSS? Apply the pseudo classes and pseudo elements on the following conditions: (i) Pseudo class and pseudo element on any two-input control (ii) Pseudo class and pseudo element on anchor and paragraph element.

Answer:

Both **pseudo-classes** and **pseudo-elements** let you style parts of a document without adding extra HTML/class attributes, but they target fundamentally different things: a pseudo-class selects an element based on its **state or position** (e.g. hovered, focused, first child), while a pseudo-element lets you style a **specific part of an element's content** that does not exist as its own DOM node (e.g. the first letter, or content inserted before/after an element).

Pseudo-class vs Pseudo-element — Comparison

Aspect	Pseudo-class	Pseudo-element
Syntax	Single colon — :hover, :focus	Double colon (CSS3) — ::before, ::after
What it targets	The state/condition of an existing element	A specific sub-part of an element's content
Creates new content?	No — only styles based on state	Yes — ::before/::after can insert generated content
Examples	:hover, :focus, :first-child, :nth-child(), :checked, :disabled	::before, ::after, ::first-line, ::first-letter, ::placeholder
Typical use	Highlighting a link on hover, styling a checked checkbox	Adding a decorative icon before text, styling only the first letter of a paragraph

(i) On a Two-Input Control (Text Box + Checkbox)

```
<!DOCTYPE html>
<html>
<head>
<style>
  /* Pseudo-class: styles the text input when it gains focus */
  input[type="text"]:focus {
    border: 2px solid #0b6e4f;
    background-color: #eaf7f0;
  }

  /* Pseudo-element: styles the placeholder text inside the input */
  input[type="text"]::placeholder {
    color: #999;
    font-style: italic;
  }

  /* Pseudo-class: styles the checkbox when it is checked */
  input[type="checkbox"]:checked {
    outline: 2px solid #0b6e4f;
  }
</style>
</head>
<body>
  <input type="text" placeholder="Enter your name">
  <br><br>
  <label><input type="checkbox"> Subscribe to newsletter</label>
</body>
</html>
```

(ii) On Anchor (<a>) and Paragraph (<p>) Elements

```
<!DOCTYPE html>
```

```
<html>
<head>
<style>
  /* Pseudo-class: styles the link only while the mouse hovers over it */
  a:hover {
    color: red;
    text-decoration: underline;
  }

  /* Pseudo-element: inserts an arrow before every link automatically */
  a::before {
    content: "-> ";
    color: #0b6e4f;
  }

  /* Pseudo-class: styles the first paragraph on the page differently */
  p:first-child {
    font-weight: bold;
  }

  /* Pseudo-element: enlarges only the first letter of every paragraph */
  p::first-letter {
    font-size: 200%;
    color: #b00020;
  }
</style>
</head>
<body>
  <p>This is the first paragraph and it is bold, with a large red first letter.</p>
  <p>This is a second, normal paragraph, also with a large first letter.</p>
  <a href="#">Hover over this link to see it turn red</a>
</body>
</html>
```

Explain the concept of box model in CSS with proper diagram, and demonstrate with example how do its components (content, padding, border, and margin) affect the layout and spacing of elements on a web page?

Answer:

Every element in CSS is treated as a rectangular box, and the **CSS Box Model** describes how the total size of that box is calculated from four concentric layers: content, padding, border, and margin, from innermost to outermost.

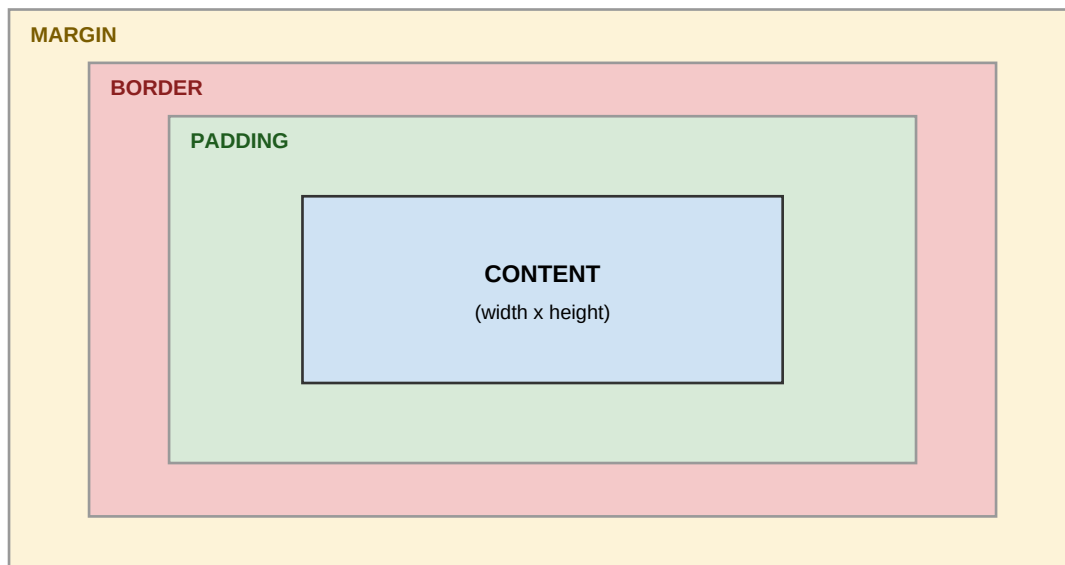


Fig. 1 — The CSS Box Model: content, padding, border, and margin

The Four Components

Component	Effect on Layout & Spacing
Content	The actual text/image/inner elements of the box; its size is set by the width and height properties.
Padding	Transparent space between the content and the border; increases the box's visible size and pushes the border outward, without affecting neighbouring elements.
Border	A visible (or invisible) line drawn around the padding; adds to the box's total rendered size unless box-sizing: border-box is used.
Margin	Transparent space outside the border; creates spacing between this box and neighbouring elements, and does not have a background color.

Total Box Size Formula

Total width = margin-left + border-left + padding-left
 + content width
 + padding-right + border-right + margin-right

Total height = margin-top + border-top + padding-top
 + content height
 + padding-bottom + border-bottom + margin-bottom

Example Demonstrating the Box Model

```
<!DOCTYPE html>
<html>
<head>
```

```
<style>
  .box {
    width: 200px;          /* content width */
    height: 100px;         /* content height */
    padding: 20px;         /* space inside the border */
    border: 5px solid #2c3e50; /* visible border */
    margin: 30px;          /* space outside the border */
    background-color: #cfe2f3;
  }
</style>
</head>
<body>
  <div class="box">Box Model Example</div>
</body>
</html>
```

Here the content area is 200x100px, but padding (20px all sides) adds 40px to each dimension, the border (5px) adds another 10px, so the box's **visible** rendered size becomes 260x150px. The margin (30px) then pushes any neighbouring element another 30px away on every side — increasing padding enlarges the box itself, while increasing margin only increases the gap to elements around it.

By default, `box-sizing` is `content-box` (width/height apply to content only, as above). Setting `box-sizing: border-box`; instead makes the specified width/height include padding and border, which is often more predictable for layout work.

Write a program to change the height, width, border and background-color of a div element when the mouse hover it.

Answer:

The :hover pseudo-class applies a set of styles to an element only while the mouse pointer is over it, reverting automatically once the pointer moves away. Combined with a CSS transition, the change can also animate smoothly instead of snapping instantly.

```
<!DOCTYPE html>
<html>
<head>
<style>
  .card {
    width: 150px;
    height: 100px;
    border: 2px solid #2c3e50;
    background-color: #cfe2f3;
    text-align: center;
    line-height: 100px;
    font-family: Arial, sans-serif;
    transition: all 0.4s ease; /* smooth animated change */
  }

  /* Styles applied only while the mouse is hovering the div */
  .card:hover {
    width: 220px;
    height: 150px;
    border: 4px solid #b00020;
    background-color: #ffe082;
    line-height: 150px;
  }
</style>
</head>
<body>
  <div class="card">Hover over me</div>
</body>
</html>
```

On hover, the div grows from 150x100px to 220x150px, its border widens from 2px solid #2c3e50 to 4px solid #b00020, and its background changes from light blue to light amber; the transition property makes all of these changes animate over 0.4 seconds instead of jumping instantly.

Write the HTML code to create the following: (a) A text box named user (b) A group of radio buttons for favorite month (c) A password box (d) A select list for favorite social networking site (e) An image called *signup.gif* as an image button on a form (f) A fieldset and legend with the text 'shipping address' around address line, city, state, pin code.

Answer:

All six form control requirements are combined below into a single HTML form, each labelled and commented according to the part of the question it answers.

```
<!DOCTYPE html>
<html>
<head><title>Form Controls Demo</title></head>
<body>

<form action="/submit" method="post">

  <!-- (a) Text box named "user" -->
  <label for="user">Your Name:</label>
  <input type="text" id="user" name="user"><br><br>

  <!-- (b) Group of radio buttons - favourite month -->
  <p>Favourite Month:</p>
  <input type="radio" id="jan" name="favmonth" value="January">
  <label for="jan">January</label><br>
  <input type="radio" id="jun" name="favmonth" value="June">
  <label for="jun">June</label><br>
  <input type="radio" id="dec" name="favmonth" value="December">
  <label for="dec">December</label><br><br>

  <!-- (c) Password box -->
  <label for="pwd">Password:</label>
  <input type="password" id="pwd" name="pwd"><br><br>

  <!-- (d) Select list - favourite social networking site -->
  <label for="social">Favourite Social Networking Site:</label>
  <select id="social" name="social">
    <option value="instagram">Instagram</option>
    <option value="facebook">Facebook</option>
    <option value="linkedin">LinkedIn</option>
    <option value="twitter">X (Twitter)</option>
  </select><br><br>

  <!-- (e) Image as an image button on the form -->
  <input type="image" src="signup.gif" alt="Sign Up" width="120"><br><br>

  <!-- (f) Fieldset and legend "shipping address" -->
  <fieldset>
    <legend>Shipping Address</legend>

    <label for="addr">Address Line:</label>
    <input type="text" id="addr" name="addr"><br><br>

    <label for="city">City:</label>
    <input type="text" id="city" name="city"><br><br>

    <label for="state">State:</label>
    <input type="text" id="state" name="state"><br><br>

    <label for="pin">Pin Code:</label>
    <input type="text" id="pin" name="pin"><br><br>
  </fieldset>

</form>
```

```
</body>
</html>
```

Notes on the Controls Used

Part	Control / Attribute Used
(a)	<code><input type="text" name="user"></code> — a single-line text box named 'user'.
(b)	Multiple <code><input type="radio"></code> elements sharing the same name ("favmonth") so only one can be selected at a time.
(c)	<code><input type="password"></code> — masks the characters the user types.
(d)	<code><select></code> with nested <code><option></code> elements to create a dropdown list.
(e)	<code><input type="image" src="signup.gif"></code> — renders the image itself as a clickable submit button.
(f)	<code><fieldset></code> groups related controls visually with a border; <code><legend></code> provides the caption 'Shipping Address' for that group.